DATA STRUCTURES AND ALGORITHMS · 2026 ASSIGNMENT

Shortest Routes and Minimum Spanning Tree on the Blantyre City Road Network, from Limbe to Blantyre Town

A weighted graph of 23 places and 37 roads in Blantyre city, solved with Dijkstra's algorithm, Kruskal's algorithm and Prim's algorithm, implemented in C++ and deployed as a web application.

Group members

#	Name	Registration number
1	[Member 1 full name]	[Registration number]
2	[Member 2 full name]	[Registration number]
3	[Member 3 full name]	[Registration number]

Source code	https://github.com/mse23-mkapolo-cpu/DSA
Live demo	https://mse23-mkapolo-cpu.github.io/DSA/
Date	November 2026

Contents

1. The graph: Blantyre city road network (Q1)
2. Drawing and description of the graph (Q2)
3. How Dijkstra's algorithm works on the graph (Q3)
4. Implementation (Q4)
5. Testing and dry run (Q5)
6. Does the graph have an MST? (Q6)
7. Which MST algorithm and why (Q7)
8. Deployment (Q8)
9. Appendix: source code

1. The graph: Blantyre city road network

Q1 · 15 marks

Scenario. The graph models the roads of Blantyre city, from Blantyre town (the city centre) in the west to Limbe in the east. Each **vertex** is a place in the city: a neighbourhood (for example Ndirande, Chilomoni, Soche, Chinyonga), a landmark (Queen Elizabeth Central Hospital, Chichiri) or a major junction (Ginnery Corner). Each **edge** is a road that links two places directly, without passing through another place in the graph. The **weight** of an edge is the road distance between the two places.

Blantyre and Limbe are joined by two main roads, and both appear in the graph. Masauko Chipembere Highway runs from the city centre past Ginnery Corner and Kanjedza to Limbe. Kamuzu Highway runs past Chichiri and Chinyonga. Smaller roads connect the residential areas around them.

The graph is **undirected** because the roads carry traffic in both directions, so the distance from A to B equals the distance from B to A. It is **weighted** because the roads have different lengths, and it is **simple** (no self loops, at most one road between two places).

Distances are approximate road distances, rounded to the nearest 100 m. They are stored in the data file as whole metres (for example 1500), and the program shows them in kilometres with one decimal (1.5 km). Whole numbers avoid rounding errors when distances are added.

23	37	78.0	9.1
vertices (places)	edges (roads)	km of road in total	km, shortest route Blantyre CBD to Limbe

Formally $G = (V, E, w)$ with $|V| = 23$, $|E| = 37$ and $w : E \to \mathbb{N}$ (metres). The full data is in `data/blantyre_roads.csv` in the repository, one road per line as `place_a,place_b,metres`.

Vertices and adjacency list

ID	Place	Area	Degree	Neighbours (distance in km)
0	Blantyre CBD City centre (Victoria Avenue, Haile Selassie Road)	Blantyre town	6	Mandala (1.5), Chilomoni (2.5), Nyambadwe (1.8), Sunnyside (1.3), Namiwawa (2.0), QECH (2.2)
1	Mandala Mandala House area	Blantyre town	3	Blantyre CBD (1.5), QECH (1.0), Naperi (2.0)
2	Chilomoni	Blantyre west	3	Blantyre CBD (2.5), Kameza (2.0), Manja (2.6)
3	Nyambadwe	Blantyre north	3	Blantyre CBD (1.8), Kameza (2.4), Ndirande (2.5)
4	Sunnyside	Blantyre town	2	Blantyre CBD (1.3), Soche (1.8)
5	Namiwawa	Blantyre north	3	Blantyre CBD (2.0), Ginnery Corner (1.6), Ndirande (2.2)
6	QECH Queen Elizabeth Central Hospital	Blantyre town	4	Blantyre CBD (2.2), Mandala (1.0), Chichiri (1.2), Ginnery Corner (1.5)
7	Kameza	Blantyre west	2	Chilomoni (2.0), Nyambadwe (2.4)

ID	Place	Area	Degree	Neighbours (distance in km)
8	Manja	Blantyre west	2	Chilomoni (2.6), Zingwangwa (2.3)
9	Zingwangwa	Blantyre west	2	Manja (2.3), Soche (1.6)
10	Soche	Blantyre south	3	Zingwangwa (1.6), Sunnyside (1.8), Naperi (1.5)
11	Naperi	Blantyre south	3	Soche (1.5), Chitawira (1.7), Mandala (2.0)
12	Chitawira	Middle	3	Naperi (1.7), Chichiri (1.4), Chinyonga (3.4)
13	Chichiri Chichiri Shopping Mall, Kamuzu Stadium	Middle	4	QECH (1.2), Ginnery Corner (1.3), Chitawira (1.4), Chinyonga (3.2)
14	Ginnery Corner Major junction on Masauko Chipembere Highway	Middle	5	QECH (1.5), Namiwawa (1.6), Makata (1.4), Chichiri (1.3), Kanjedza (3.0)
15	Ndirande	Middle	4	Nyambadwe (2.5), Namiwawa (2.2), Makata (2.0), Machinjiri (4.5)
16	Makata Industrial area	Middle	3	Ndirande (2.0), Ginnery Corner (1.4), Kanjedza (2.8)
17	Kanjedza	Limbe side	4	Ginnery Corner (3.0), Makata (2.8), Chinyonga (1.5), Limbe (2.5)
18	Chinyonga	Limbe side	4	Chichiri (3.2), Chitawira (3.4), Kanjedza (1.5), Mpingwe (1.8)
19	Limbe Limbe town centre and market	Limbe side	4	Kanjedza (2.5), Mpingwe (1.2), Bangwe (2.8), Machinjiri (3.5)
20	Mpingwe	Limbe side	3	Chinyonga (1.8), Limbe (1.2), Bangwe (2.5)
21	Bangwe	Limbe side	2	Limbe (2.8), Mpingwe (2.5)
22	Machinjiri	Limbe side	2	Limbe (3.5), Ndirande (4.5)

Edges (weights in km)

Edge	Place A	Place B	km
e1	Blantyre CBD	Mandala	1.5
e2	Blantyre CBD	Chilomoni	2.5
e3	Blantyre CBD	Nyambadwe	1.8
e4	Blantyre CBD	Sunnyside	1.3
e5	Blantyre CBD	Namiwawa	2.0
e6	Blantyre CBD	QECH	2.2
e7	Chilomoni	Kameza	2.0
e8	Kameza	Nyambadwe	2.4
e9	Chilomoni	Manja	2.6
e10	Manja	Zingwangwa	2.3
e11	Zingwangwa	Soche	1.6

Edge	Place A	Place B	km
e20	Nyambadwe	Ndirande	2.5
e21	Namiwawa	Ndirande	2.2
e22	Ndirande	Makata	2.0
e23	Ginnery Corner	Makata	1.4
e24	Ginnery Corner	Chichiri	1.3
e25	Chichiri	Chitawira	1.4
e26	Ginnery Corner	Kanjedza	3.0
e27	Makata	Kanjedza	2.8
e28	Chichiri	Chinyonga	3.2
e29	Chitawira	Chinyonga	3.4

Edge	Place A	Place B	km
e12	Sunnyside	Soche	1.8
e13	Soche	Naperi	1.5
e14	Naperi	Chitawira	1.7
e15	Mandala	QECH	1.0
e16	Mandala	Naperi	2.0
e17	QECH	Chichiri	1.2
e18	QECH	Ginnery Corner	1.5
e19	Namiwawa	Ginnery Corner	1.6

Edge	Place A	Place B	km
e30	Kanjedza	Chinyonga	1.5
e31	Kanjedza	Limbe	2.5
e32	Chinyonga	Mpingwe	1.8
e33	Mpingwe	Limbe	1.2
e34	Limbe	Bangwe	2.8
e35	Mpingwe	Bangwe	2.5
e36	Limbe	Machinjiri	3.5
e37	Ndirande	Machinjiri	4.5

2. Drawing and description of the graph

Q2 · 10 marks

Conventions used. Each place is drawn as a circle (vertex) labelled with its name. Each road is a straight line (edge) between two circles. Because the graph is undirected, the lines have no arrowheads. The number in the box on each line is the weight, the distance in km. Places are laid out roughly as they are on a map of Blantyre, with Blantyre town on the left (west), Limbe on the right (east) and north at the top.

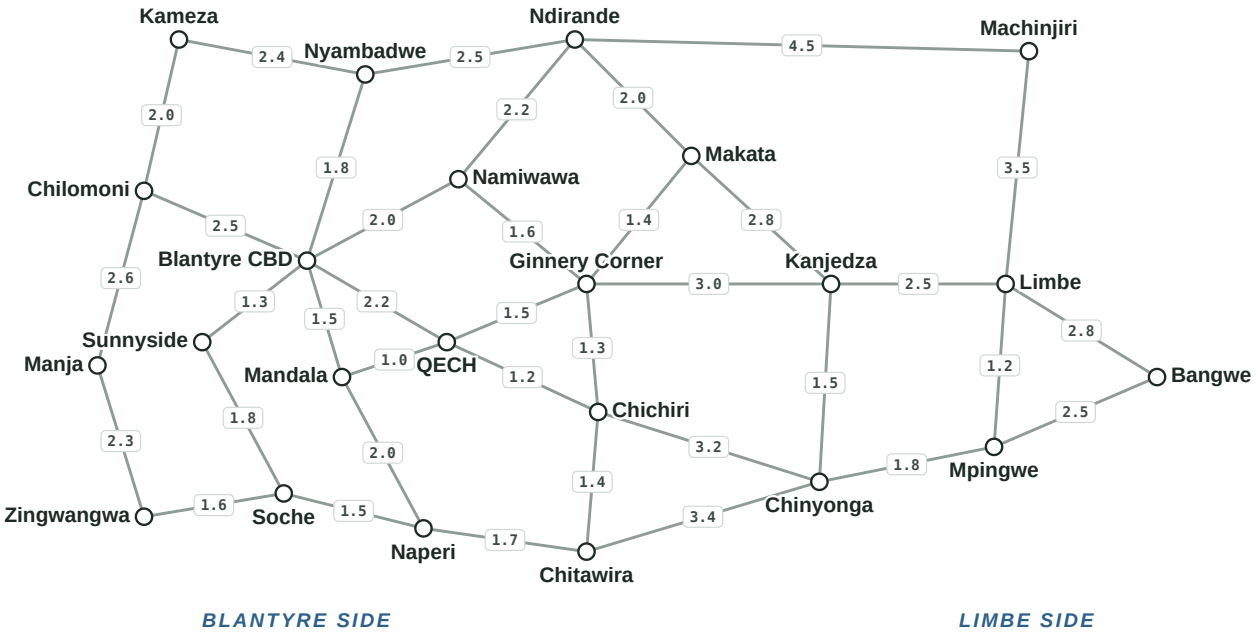


Figure 1. The Blantyre city road network graph, G = (V, E), |V| = 23, |E| = 37. Weights in km.

Description

- **Connected:** there is a path between every pair of places, so G has a single component. Test T10 confirms this: all 529 source–target pairs have a finite distance.
- **Sparse:** a simple graph on 23 vertices can have at most 23 × 22 / 2 = 253 edges. G has 37, a density of 37 / 253 ≈ 0.15. This is typical of road networks and is why an adjacency list is the right representation.
- **Degrees:** the sum of degrees is 2|E| = 74 (handshake lemma), so the average degree is 3.22. Blantyre CBD has the highest degree (6), as the city centre where many roads meet. Ginnery Corner, the busy junction between Blantyre and Limbe, has degree 5. 6 places have degree 2: Sunnyside, Kameza, Manja, Zingwangwa, Bangwe, Machinjiri. These are at the edge of the city.
- **Cycles:** G contains many cycles, for example Blantyre CBD–Mandala–QECH–Blantyre CBD (1.5 + 1.0 + 2.2 = 4.7 km) and the two highway corridors Ginnery Corner–Kanjedza–Chinyonga–Chichiri–Ginnery Corner. The cyclomatic number |E| − |V| + 1 = 15 is the number of independent cycles, and also the number of roads that can be removed while keeping all places connected (see the MST in section 6).
- **No bridges:** removing any single road leaves the graph connected, because every road lies on a cycle. In practice, if one road is closed (for example for repairs) every place can still be reached by another route.

- **Longest and shortest roads:** Ndirande–Machinjiri (4.5 km) is the longest. Mandala–QECH (1.0 km) is the shortest.

> The assignment asks for the graph to be drawn on paper. Figure 1 is the typed version; the hand-drawn version follows the same layout and conventions.

3. How Dijkstra's algorithm works on the graph

Q3 · 15 marks

Dijkstra's algorithm finds the shortest distance from one **source** vertex to every other vertex in a graph with non-negative weights. It keeps a tentative distance *d[v]* for every place, starting at 0 for the source and ∞ for the rest. At each step it **settles** the unsettled place with the smallest tentative distance; that distance is now final, because every other route would have to leave through a place that is already at least as far away and the weights are never negative. It then **relaxes** each road (u, v) out of the settled place: if d[u] + w(u, v) < d[v], it lowers d[v] and records u as the previous place on v's route.

```
DIJKSTRA(G, s)
  for each vertex v:  d[v] ← ∞,  prev[v] ← NIL
  d[s] ← 0;  Q ← min-priority queue containing (0, s)
  while Q is not empty:
      (du, u) ← EXTRACT-MIN(Q)
      if u is already settled: continue          // stale entry
      mark u settled
      for each edge (u, v, w):
          if v not settled and du + w < d[v]:
              d[v] ← du + w;  prev[v] ← u;  INSERT(Q, (d[v], v))
```

Worked example: all shortest routes from Blantyre CBD

We run the algorithm with Blantyre CBD as the source. The table shows, for each of the 23 steps, which place is settled, the roads that improved a tentative distance, and the reached-but-unsettled places afterwards (in order of distance, all in km). A place leaves the queue in exactly the order of its final distance.

Step	Settle	d (km)	Relaxations (old → new)	Tentative distances after the step
1	**Blantyre CBD**	0.0	Mandala: ∞ → **1.5** Chilomoni: ∞ → **2.5** Nyambadwe: ∞ → **1.8** Sunnyside: ∞ → **1.3** Namiwawa: ∞ → **2.0** QECH: ∞ → **2.2**	Sunnyside 1.3, Mandala 1.5, Nyambadwe 1.8, Namiwawa 2.0, QECH 2.2, Chilomoni 2.5
2	**Sunnyside**	1.3	Soche: ∞ → **3.1**	Mandala 1.5, Nyambadwe 1.8, Namiwawa 2.0, QECH 2.2, Chilomoni 2.5, Soche 3.1
3	**Mandala**	1.5	Naperi: ∞ → **3.5**	Nyambadwe 1.8, Namiwawa 2.0, QECH 2.2, Chilomoni 2.5, Soche 3.1, Naperi 3.5
4	**Nyambadwe**	1.8	Kameza: ∞ → **4.2** Ndirande: ∞ → **4.3**	Namiwawa 2.0, QECH 2.2, Chilomoni 2.5, Soche 3.1, Naperi 3.5, Kameza 4.2, Ndirande 4.3
5	**Namiwawa**	2.0	Ginnery Corner: ∞ → **3.6** Ndirande: 4.3 → **4.2**	QECH 2.2, Chilomoni 2.5, Soche 3.1, Naperi 3.5, Ginnery Corner 3.6, Kameza 4.2, Ndirande 4.2
6	**QECH**	2.2	Chichiri: ∞ → **3.4**	Chilomoni 2.5, Soche 3.1, Chichiri 3.4, Naperi 3.5, Ginnery Corner 3.6, Kameza 4.2, Ndirande 4.2
7	**Chilomoni**	2.5	Manja: ∞ → **5.1**	Soche 3.1, Chichiri 3.4, Naperi 3.5, Ginnery Corner 3.6, Kameza 4.2, Ndirande 4.2, Manja 5.1
8	**Soche**	3.1	Zingwangwa: ∞ → **4.7**	Chichiri 3.4, Naperi 3.5, Ginnery Corner 3.6, Kameza 4.2, Ndirande 4.2, Zingwangwa 4.7, Manja 5.1

Step	Settle	d (km)	Relaxations (old → new)	Tentative distances after the step
9	**Chichiri**	3.4	Chitawira: ∞ → **4.8** Chinyonga: ∞ → **6.6**	Naperi 3.5, Ginnery Corner 3.6, Kameza 4.2, Ndirande 4.2, Zingwangwa 4.7, Chitawira 4.8, Manja 5.1, Chinyonga 6.6
10	**Naperi**	3.5	none (no shorter road found)	Ginnery Corner 3.6, Kameza 4.2, Ndirande 4.2, Zingwangwa 4.7, Chitawira 4.8, Manja 5.1, Chinyonga 6.6
11	**Ginnery Corner**	3.6	Makata: ∞ → **5.0** Kanjedza: ∞ → **6.6**	Kameza 4.2, Ndirande 4.2, Zingwangwa 4.7, Chitawira 4.8, Makata 5.0, Manja 5.1, Kanjedza 6.6, Chinyonga 6.6
12	**Kameza**	4.2	none (no shorter road found)	Ndirande 4.2, Zingwangwa 4.7, Chitawira 4.8, Makata 5.0, Manja 5.1, Kanjedza 6.6, Chinyonga 6.6
13	**Ndirande**	4.2	Machinjiri: ∞ → **8.7**	Zingwangwa 4.7, Chitawira 4.8, Makata 5.0, Manja 5.1, Kanjedza 6.6, Chinyonga 6.6, Machinjiri 8.7
14	**Zingwangwa**	4.7	none (no shorter road found)	Chitawira 4.8, Makata 5.0, Manja 5.1, Kanjedza 6.6, Chinyonga 6.6, Machinjiri 8.7
15	**Chitawira**	4.8	none (no shorter road found)	Makata 5.0, Manja 5.1, Kanjedza 6.6, Chinyonga 6.6, Machinjiri 8.7
16	**Makata**	5.0	none (no shorter road found)	Manja 5.1, Kanjedza 6.6, Chinyonga 6.6, Machinjiri 8.7
17	**Manja**	5.1	none (no shorter road found)	Kanjedza 6.6, Chinyonga 6.6, Machinjiri 8.7
18	**Kanjedza**	6.6	Limbe: ∞ → **9.1**	Chinyonga 6.6, Machinjiri 8.7, Limbe 9.1
19	**Chinyonga**	6.6	Mpingwe: ∞ → **8.4**	Mpingwe 8.4, Machinjiri 8.7, Limbe 9.1
20	**Mpingwe**	8.4	Bangwe: ∞ → **10.9**	Machinjiri 8.7, Limbe 9.1, Bangwe 10.9
21	**Machinjiri**	8.7	none (no shorter road found)	Limbe 9.1, Bangwe 10.9
22	**Limbe**	9.1	none (no shorter road found)	Bangwe 10.9
23	**Bangwe**	10.9	none (no shorter road found)	(empty)

Step 5 shows the algorithm correcting itself. In step 4, Nyambadwe sets Ndirande to 1.8 + 2.5 = 4.3 km. In step 5, Namiwawa (2.0 km) finds a shorter road, 2.0 + 2.2 = 4.2 km, so Ndirande is updated to 4.2 and its previous place changes to Namiwawa. Some steps, such as Naperi and Kameza, change nothing because every neighbour already has a shorter distance.

Result: shortest distances and routes from Blantyre CBD

Place	Distance (km)	Previous place	Shortest route
Blantyre CBD	0.0	–	Blantyre CBD
Sunnyside	1.3	Blantyre CBD	Blantyre CBD → Sunnyside
Mandala	1.5	Blantyre CBD	Blantyre CBD → Mandala
Nyambadwe	1.8	Blantyre CBD	Blantyre CBD → Nyambadwe
Namiwawa	2.0	Blantyre CBD	Blantyre CBD → Namiwawa
QECH	2.2	Blantyre CBD	Blantyre CBD → QECH

Place	Distance (km)	Previous place	Shortest route
Chilomoni	2.5	Blantyre CBD	Blantyre CBD → Chilomoni
Soche	3.1	Sunnyside	Blantyre CBD → Sunnyside → Soche
Chichiri	3.4	QECH	Blantyre CBD → QECH → Chichiri
Naperi	3.5	Mandala	Blantyre CBD → Mandala → Naperi
Ginnery Corner	3.6	Namiwawa	Blantyre CBD → Namiwawa → Ginnery Corner
Kameza	4.2	Nyambadwe	Blantyre CBD → Nyambadwe → Kameza
Ndirande	4.2	Namiwawa	Blantyre CBD → Namiwawa → Ndirande
Zingwangwa	4.7	Soche	Blantyre CBD → Sunnyside → Soche → Zingwangwa
Chitawira	4.8	Chichiri	Blantyre CBD → QECH → Chichiri → Chitawira
Makata	5.0	Ginnery Corner	Blantyre CBD → Namiwawa → Ginnery Corner → Makata
Manja	5.1	Chilomoni	Blantyre CBD → Chilomoni → Manja
Kanjedza	6.6	Ginnery Corner	Blantyre CBD → Namiwawa → Ginnery Corner → Kanjedza
Chinyonga	6.6	Chichiri	Blantyre CBD → QECH → Chichiri → Chinyonga
Mpingwe	8.4	Chinyonga	Blantyre CBD → QECH → Chichiri → Chinyonga → Mpingwe
Machinjiri	8.7	Ndirande	Blantyre CBD → Namiwawa → Ndirande → Machinjiri
Limbe	9.1	Kanjedza	Blantyre CBD → Namiwawa → Ginnery Corner → Kanjedza → Limbe
Bangwe	10.9	Mpingwe	Blantyre CBD → QECH → Chichiri → Chinyonga → Mpingwe → Bangwe

The previous-place pointers form the **shortest-path tree** rooted at Blantyre CBD (Figure 2). Reading the route to any place means following the pointers back from that place to Blantyre CBD. For example, Limbe ← Kanjedza ← Ginnery Corner ← Namiwawa ← Blantyre CBD gives Blantyre CBD → Namiwawa → Ginnery Corner → Kanjedza → Limbe, 2.0 + 1.6 + 3.0 + 2.5 = **9.1 km**, along Masauko Chipembere Highway. This is shorter than the Kamuzu Highway route through QECH, Chichiri, Chinyonga and Mpingwe (2.2 + 1.2 + 3.2 + 1.8 + 1.2 = 9.6 km) and the route through Makata (2.0 + 1.6 + 1.4 + 2.8 + 2.5 = 10.3 km).

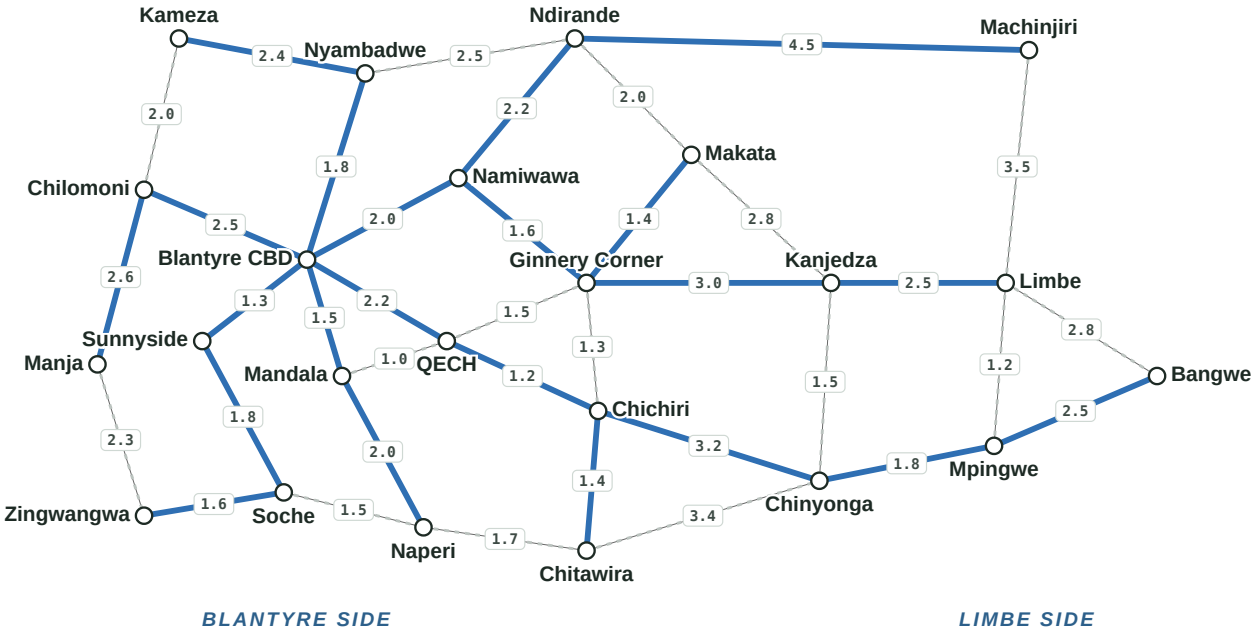


Figure 2. Shortest-path tree from Blantyre CBD (blue). Dashed roads are not used by any shortest route from Blantyre CBD.

4. Implementation

Q4 · 10 marks

The algorithm is implemented in **C++17** with only the standard library. The repository is organised as follows:

File	Purpose
`data/blantyre_roads.csv`	The graph: one road per line, `place_a,place_b,metres`.
`src/graph.hpp`	Graph class (adjacency list), Dijkstra, path reconstruction, Kruskal (with union–find) and Prim.
`src/main.cpp`	Command line program: shortest route, step-by-step trace, distances from a place, MST, and an interactive menu.
`tests/tests.cpp`	25 automated tests (section 5).
`docs/`	Web version of the same algorithms in JavaScript, published with GitHub Pages (section 8).
`Makefile`	`make` builds everything, `make test` runs the tests.

Design decisions

- **Adjacency list** (`vector<vector<pair<int, long long>>>`): uses O(V + E) memory and lets Dijkstra visit only the real neighbours of a place. An adjacency matrix would need 23 × 23 = 529 cells for 37 roads.
- **Place names are mapped to integer IDs** with a `std::map`, so the algorithms work on fast array indices while the user types names.
- **Distances in whole metres.** Storing 1.5 km as 1500 m keeps all arithmetic in integers, so sums are exact and comparisons such as d[u] + w < d[v] never suffer from floating-point rounding. Output is converted to km.
- **Binary heap priority queue** (`std::priority_queue` with `std::greater`) holding (distance, place) pairs. Instead of a decrease-key operation, a place is pushed again when its distance improves, and outdated entries are skipped when popped (“lazy deletion”). This keeps the code short and gives **O((V + E) log V)** time.
- **Path reconstruction** uses the `prev[]` array, following pointers back from the target and reversing.
- **Input validation:** negative weights are rejected (Dijkstra is incorrect with them), as are self loops, malformed lines, non-numeric distances, missing files and unknown place names. Unreachable places are reported as “No route”.

Core code: Dijkstra

```
struct DijkstraResult {
    std::vector<long long> dist;  // dist[v] = shortest distance in metres from source (INF if
unreachable)
    std::vector<int> prev;        // prev[v] = previous place on the shortest path (-1 if none)
};

// One step of the algorithm, recorded for the dry run trace.
struct TraceStep {
    int settled;                                  // place removed from the queue and finalised
    long long settledDist;
    std::vector<std::tuple<int, long long, long long>> relaxed;  // (place, old dist, new dist)
};
```

```
// Dijkstra with a binary heap (std::priority_queue): O((V + E) log V).
inline DijkstraResult dijkstra(const Graph& g, int source, std::vector<TraceStep>* trace = nullptr) {
    int n = g.size();
    if (source < 0 || source >= n) throw std::out_of_range("Source index out of range");
    DijkstraResult r{std::vector<long long>(n, INF), std::vector<int>(n, -1)};
    std::vector<bool> done(n, false);
    using Item = std::pair<long long, int>;  // (distance, place)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;

    r.dist[source] = 0;
    pq.push({0, source});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (done[u]) continue;  // stale queue entry
        done[u] = true;
        TraceStep step{u, d, {}};
        for (auto [v, w] : g.neighbours(u)) {
            if (done[v]) continue;
            if (d + w < r.dist[v]) {
                step.relaxed.emplace_back(v, r.dist[v], d + w);
                r.dist[v] = d + w;
                r.prev[v] = u;
                pq.push({r.dist[v], v});
            }
        }
        if (trace) trace->push_back(step);
    }
    return r;
}

// Rebuild the path source -> target from prev[]. Empty if unreachable.
inline std::vector<int> buildPath(const DijkstraResult& r, int target) {
    std::vector<int> path;
    if (r.dist.at(target) >= INF) return path;
    for (int v = target; v != -1; v = r.prev[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
    return path;
}
```

Building and running

```
$ make                       # builds ./roads and ./run_tests
$ ./roads path "Blantyre CBD" Limbe
Shortest route Blantyre CBD -> Limbe: 9.1 km
  Blantyre CBD -> Namiwawa -> Ginnery Corner -> Kanjedza -> Limbe
$ ./roads trace Mandala Chichiri    # step-by-step dry run
$ ./roads all Limbe                 # distances from Limbe to every place
$ ./roads mst kruskal               # minimum spanning tree
$ ./roads                           # interactive menu
$ make test                         # run the 25 tests
```

5. Testing and dry run

Q5 · 20 marks

Test strategy

The tests in `tests/tests.cpp` cover four groups: (a) loading the graph, (b) known shortest routes worked out by hand, (c) an independent cross-check of every pair of places against a different algorithm (Floyd–Warshall), and (d) error handling and edge cases. Each test prints its input, the expected output and the actual output. The table below was produced by running `make test`.

Summary: 25 passed, 0 failed

ID	Test	Input	Expected	Actual	Result
T01	place and road count	data/blantyre_roads.csv	`23 places, 37 roads`	`23 places, 37 roads`	PASS
T02	same start and end	Mandala → Mandala	`0.0 km: Mandala`	`0.0 km: Mandala`	PASS
T03	direct neighbours	Blantyre CBD → Mandala	`1.5 km: Blantyre CBD > Mandala`	`1.5 km: Blantyre CBD > Mandala`	PASS
T04	Blantyre town to Limbe	Blantyre CBD → Limbe	`9.1 km: Blantyre CBD > Namiwawa > Ginnery Corner > Kanjedza > Limbe`	`9.1 km: Blantyre CBD > Namiwawa > Ginnery Corner > Kanjedza > Limbe`	PASS
T05	route through residential areas	Soche → Chichiri	`4.6 km: Soche > Naperi > Chitawira > Chichiri`	`4.6 km: Soche > Naperi > Chitawira > Chichiri`	PASS
T06	far west to far east	Kameza → Bangwe	`15.0 km: Kameza > Nyambadwe > Ndirande > Makata > Kanjedza > Limbe > Bangwe`	`15.0 km: Kameza > Nyambadwe > Ndirande > Makata > Kanjedza > Limbe > Bangwe`	PASS
T07	symmetric (undirected)	Bangwe → Kameza	`15.0 km`	`15.0 km`	PASS
T08	west to north-east	Manja → Machinjiri	`13.8 km: Manja > Chilomoni > Blantyre CBD > Namiwawa > Ndirande > Machinjiri`	`13.8 km: Manja > Chilomoni > Blantyre CBD > Namiwawa > Ndirande > Machinjiri`	PASS
T09	south-west to Limbe side	Zingwangwa → Mpingwe	`10.0 km: Zingwangwa > Soche > Naperi > Chitawira > Chinyonga > Mpingwe`	`10.0 km: Zingwangwa > Soche > Naperi > Chitawira > Chinyonga > Mpingwe`	PASS
T10	all pairs agree, all paths add up	every source and target	`0 mismatches`	`0 mismatches`	PASS
T11	unknown place	Blantyre CBD → Lilongwe	`error: Unknown place: Lilongwe`	`error: Unknown place: Lilongwe`	PASS
T12	place names are case sensitive	limbe → Mandala	`error: Unknown place: limbe`	`error: Unknown place: limbe`	PASS
T13	negative distance rejected	addEdge(A, B, -500)	`error: Negative distance on road A - B: Dijkstra needs non-negative weights`	`error: Negative distance on road A - B: Dijkstra needs non-negative weights`	PASS

ID	Test	Input	Expected	Actual	Result
T14	distance not in whole metres	Limbe,Bangwe,2.8km	`error: Line 1: bad distance '2.8km'`	`error: Line 1: bad distance '2.8km'`	PASS
T15	wrong separator in file	Limbe;Bangwe;2800	`error: Line 1: expected place_a,place_b,metres`	`error: Line 1: expected place_a,place_b,metres`	PASS
T16	missing data file	no_such_file.csv	`error: Cannot open no_such_file.csv`	`error: Cannot open no_such_file.csv`	PASS
T17	unreachable place	Limbe → Chileka	`unreachable`	`unreachable`	PASS
T18	MST of disconnected graph is a forest	{Limbe-Bangwe 2800, Chileka-Lunzu 9000}	`2 edges, total 11.8 km, spanning=false`	`2 edges, total 11.8 km, spanning=false`	PASS
T19	zero-length road allowed	A-B 0, B-C 400, A-C 900; A → C	`0.4 km: A > B > C`	`0.4 km: A > B > C`	PASS
T20	parallel roads: shorter one used	A-B 1000 and A-B 300; A → B	`0.3 km: A > B`	`0.3 km: A > B`	PASS
T21	Kruskal MST size	Blantyre graph	`22 edges, spanning=true`	`22 edges, spanning=true`	PASS
T22	Kruskal MST total	Blantyre graph	`39.3 km`	`39.3 km`	PASS
T23	Prim gives the same total	Blantyre graph, start at Blantyre CBD	`39.3 km`	`39.3 km`	PASS
T24	Prim from every start place	23 different start places	`0 mismatches`	`0 mismatches`	PASS
T25	MST connects all places	Kruskal edges	`1 component`	`1 component`	PASS

How the expected values were checked

Each expected route in T03–T09 was worked out by hand and compared against the other possible routes. For T04 (Blantyre CBD → Limbe) we compared three corridors: Masauko Chipembere Highway via Ginnery Corner and Kanjedza (9.1 km), Kamuzu Highway via Chichiri and Chinyonga (9.6 km), and the northern route via Makata (10.3 km). Test T10 then checks every one of the 529 pairs against Floyd–Warshall, a completely different algorithm, and checks that every route the program prints adds up to the distance it reports. This guards against mistakes in the hand calculations as well as in the code.

Errors

Tests T11–T16 check that bad input produces a clear error message instead of a crash or a wrong answer. The same errors appear in the command line program:

```
$ ./roads path "Blantyre CBD" Lilongwe
Error: Unknown place: Lilongwe
$ ./roads path limbe Mandala
```

```
Error: Unknown place: limbe
$ ./roads --data no_such_file.csv path Limbe Bangwe
Error: Cannot open no_such_file.csv
```

Edge cases (T17–T20) use small separate graphs: a place with no road link to the rest (Chileka, joined only to Lunzu) is reported as unreachable, a zero-length road is allowed, and when two roads join the same places the shorter one is used. No test failed in the final run, and no crashes or incorrect results were found.

Dry run 1: Soche → Chichiri, tracing the priority queue

This dry run follows the code line by line, showing the real contents of the priority queue, including stale entries left behind by lazy deletion. Entries are written (place, distance in km) and the queue is shown sorted, smallest first.

Iter.	Action	Relaxing each road of the settled place	Priority queue after
init	–	dist[Soche] = 0, all others ∞	(Soche, 0.0)
1	pop (Soche, 0.0) settle Soche	Zingwangwa: 0.0 + 1.6 = 1.6 < ∞ → update, prev = Soche Sunnyside: 0.0 + 1.8 = 1.8 < ∞ → update, prev = Soche Naperi: 0.0 + 1.5 = 1.5 < ∞ → update, prev = Soche	(Naperi, 1.5) (Zingwangwa, 1.6) (Sunnyside, 1.8)
2	pop (Naperi, 1.5) settle Naperi	Soche: settled, skip Chitawira: 1.5 + 1.7 = 3.2 < ∞ → update, prev = Naperi Mandala: 1.5 + 2.0 = 3.5 < ∞ → update, prev = Naperi	(Zingwangwa, 1.6) (Sunnyside, 1.8) (Chitawira, 3.2) (Mandala, 3.5)
3	pop (Zingwangwa, 1.6) settle Zingwangwa	Manja: 1.6 + 2.3 = 3.9 < ∞ → update, prev = Zingwangwa Soche: settled, skip	(Sunnyside, 1.8) (Chitawira, 3.2) (Mandala, 3.5) (Manja, 3.9)
4	pop (Sunnyside, 1.8) settle Sunnyside	Blantyre CBD: 1.8 + 1.3 = 3.1 < ∞ → update, prev = Sunnyside Soche: settled, skip	(Blantyre CBD, 3.1) (Chitawira, 3.2) (Mandala, 3.5) (Manja, 3.9)
5	pop (Blantyre CBD, 3.1) settle Blantyre CBD	Mandala: 3.1 + 1.5 = 4.6 ≥ 3.5 → no change Chilomoni: 3.1 + 2.5 = 5.6 < ∞ → update, prev = Blantyre CBD Nyambadwe: 3.1 + 1.8 = 4.9 < ∞ → update, prev = Blantyre CBD Sunnyside: settled, skip Namiwawa: 3.1 + 2.0 = 5.1 < ∞ → update, prev = Blantyre CBD QECH: 3.1 + 2.2 = 5.3 < ∞ → update, prev = Blantyre CBD	(Chitawira, 3.2) (Mandala, 3.5) (Manja, 3.9) (Nyambadwe, 4.9) (Namiwawa, 5.1) (QECH, 5.3) (Chilomoni, 5.6)

Iter.	Action	Relaxing each road of the settled place	Priority queue after
6	pop (Chitawira, 3.2) settle Chitawira	Naperi: settled, skip Chichiri: 3.2 + 1.4 = 4.6 < ∞ → update, prev = Chitawira Chinyonga: 3.2 + 3.4 = 6.6 < ∞ → update, prev = Chitawira	(Mandala, 3.5) (Manja, 3.9) (Chichiri, 4.6) (Nyambadwe, 4.9) (Namiwawa, 5.1) (QECH, 5.3) (Chilomoni, 5.6) (Chinyonga, 6.6)
7	pop (Mandala, 3.5) settle Mandala	Blantyre CBD: settled, skip QECH: 3.5 + 1.0 = 4.5 < 5.3 → update, prev = Mandala Naperi: settled, skip	(Manja, 3.9) (QECH, 4.5) (Chichiri, 4.6) (Nyambadwe, 4.9) (Namiwawa, 5.1) (QECH, 5.3) (Chilomoni, 5.6) (Chinyonga, 6.6)
8	pop (Manja, 3.9) settle Manja	Chilomoni: 3.9 + 2.6 = 6.5 ≥ 5.6 → no change Zingwangwa: settled, skip	(QECH, 4.5) (Chichiri, 4.6) (Nyambadwe, 4.9) (Namiwawa, 5.1) (QECH, 5.3) (Chilomoni, 5.6) (Chinyonga, 6.6)
9	pop (QECH, 4.5) settle QECH	Blantyre CBD: settled, skip Mandala: settled, skip Chichiri: 4.5 + 1.2 = 5.7 ≥ 4.6 → no change Ginnery Corner: 4.5 + 1.5 = 6.0 < ∞ → update, prev = QECH	(Chichiri, 4.6) (Nyambadwe, 4.9) (Namiwawa, 5.1) (QECH, 5.3) (Chilomoni, 5.6) (Ginnery Corner, 6.0) (Chinyonga, 6.6)
10	pop (Chichiri, 4.6) settle Chichiri	QECH: settled, skip Ginnery Corner: 4.6 + 1.3 = 5.9 < 6.0 → update, prev = Chichiri Chitawira: settled, skip Chinyonga: 4.6 + 3.2 = 7.8 ≥ 6.6 → no change **Target settled: stop.**	(Nyambadwe, 4.9) (Namiwawa, 5.1) (QECH, 5.3) (Chilomoni, 5.6) (Ginnery Corner, 5.9) (Ginnery Corner, 6.0) (Chinyonga, 6.6)

Result: following prev[] back from Chichiri gives Soche → Naperi → Chitawira → Chichiri, a distance of **4.6 km**. The program prints the same:

```
$ ./roads path Soche Chichiri
Shortest route Soche -> Chichiri: 4.6 km
  Soche -> Naperi -> Chitawira -> Chichiri
```

Dry run 2: program trace for Mandala → Chichiri

The `trace` command prints each step the algorithm takes and stops once the destination is settled.

```
$ ./roads trace Mandala Chichiri
Dry run of Dijkstra from Mandala
Step  1: settle Mandala (0.0 km)
          update Blantyre CBD: inf -> 1.5
          update QECH: inf -> 1.0
          update Naperi: inf -> 2.0
Step  2: settle QECH (1.0 km)
          update Chichiri: inf -> 2.2
          update Ginnery Corner: inf -> 2.5
Step  3: settle Blantyre CBD (1.5 km)
          update Chilomoni: inf -> 4.0
          update Nyambadwe: inf -> 3.3
          update Sunnyside: inf -> 2.8
          update Namiwawa: inf -> 3.5
Step  4: settle Naperi (2.0 km)
          update Soche: inf -> 3.5
          update Chitawira: inf -> 3.7
```

```
Step  5: settle Chichiri (2.2 km)
          update Chitawira: 3.7 -> 3.6
          update Chinyonga: inf -> 5.4
Target Chichiri settled, so its distance is final.

Shortest route Mandala -> Chichiri: 2.2 km
  Mandala -> QECH -> Chichiri
```

6. Does the graph have a minimum spanning tree?

Q6 · 10 marks

Yes. A spanning tree is a subset of the edges that connects all the vertices with no cycles; it has exactly |V| − 1 edges. A minimum spanning tree (MST) is a spanning tree with the smallest possible total weight. An undirected graph has a spanning tree if and only if it is **connected**, and then it also has a minimum one, because there are finitely many spanning trees and one of them must have the smallest total. Our graph is undirected, weighted and connected (section 2), so it has an MST.

The MST of our graph has **22 edges** and a total length of **39.3 km**, compared with 78.0 km for all 37 roads. The other 15 roads (the cyclomatic number from section 2) each close a cycle and are not needed for connectivity.

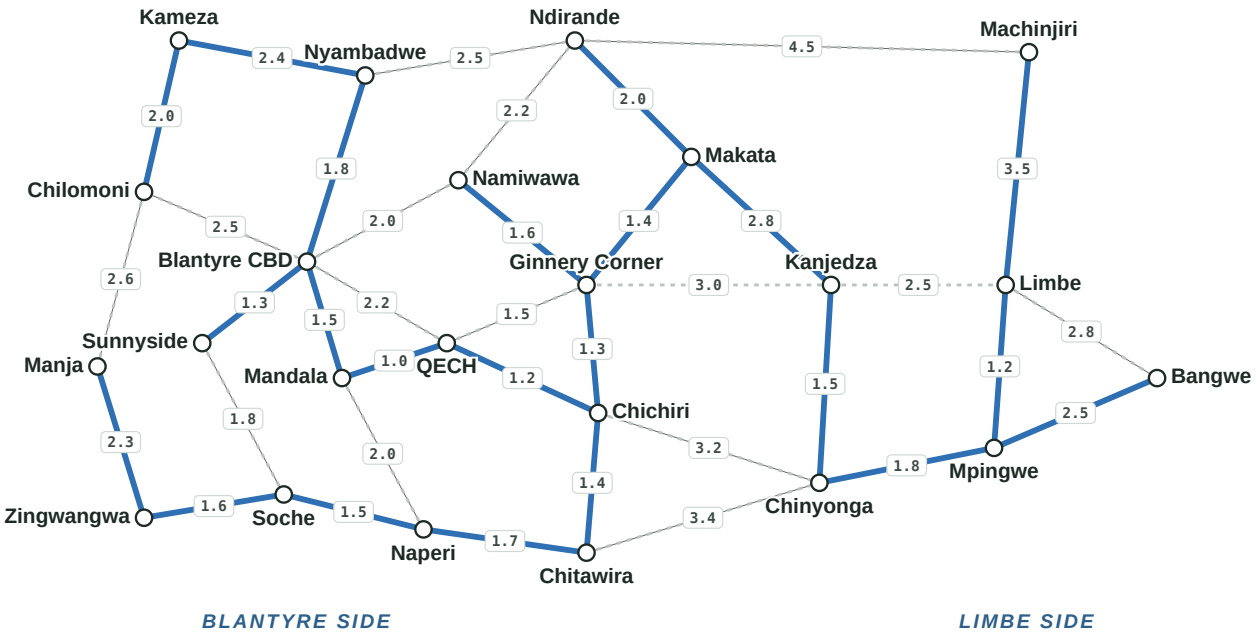


Figure 3. Minimum spanning tree (blue, 39.3 km). Dashed roads are not in the MST.

The MST is unique. Some roads have equal lengths (for example several roads of 1.5 km and of 2.5 km), which can sometimes allow more than one MST. We checked every road that is not in the tree: in each case it is strictly longer than every road on the tree path between its two places, so no swap gives an equal total. The MST is therefore the only one.

What the MST represents

The MST is the **shortest total length of road that still connects every place** to every other place. In Blantyre this answers questions such as:

- **Laying pipes or cables along the roads.** Blantyre Water Board pipes, ESCOM power lines and fibre-optic cables are usually laid along road reserves. Connecting all 23 places along the MST needs 39.3 km of pipe or cable instead of 78.0 km.
- **Road maintenance on a limited budget.** If Blantyre City Council could only resurface 22 roads, the MST keeps every area from Limbe to Blantyre town reachable for the least total length of road to maintain.
- **Emergency access.** During heavy rains, when some roads flood or wash away, the MST shows a minimal backbone that must stay open so that every area, including QECH, can still be reached.

What the MST does not represent: it is not a set of shortest routes. In the MST the only route from Blantyre CBD to Limbe is Blantyre CBD → Mandala → QECH → Chichiri → Ginnery Corner → Makata → Kanjedza → Chinyonga → Mpingwe → Limbe, which is **13.7 km**, compared with the 9.1 km found by Dijkstra. The MST minimises the total length of the whole network; Dijkstra minimises the length of one journey.

7. Which MST algorithm we would use, and why

Q7 · 10 marks

	Kruskal	Prim
Idea	Sort all roads by length. Add each road unless it joins two places that are already connected (it would make a cycle).	Start from one place. Repeatedly add the shortest road from the tree to a place not yet in the tree.
Data structure	Sorted edge list + union–find (disjoint sets)	Priority queue + adjacency list
Time	O(E log E)	O(E log V) with a binary heap; $O(V^2)$ with an array
Best for	Sparse graphs, data given as an edge list	Dense graphs, data given as an adjacency matrix
On our graph	$37 \times \log_2 37 \approx 193$ basic steps	$37 \times \log_2 23 \approx 167$ basic steps

We would use Kruskal's algorithm, for these reasons:

1. **The graph is sparse** (37 edges, density 0.15). Kruskal's cost depends on sorting the edges, which is cheap when there are few of them. The $O(V^2)$ array version of Prim, which suits dense graphs, would waste work here. For a graph this small the binary-heap versions of both algorithms are practically equal in speed, so the decision comes down to the next points.
2. **Our data is already an edge list.** The file `blantyre_roads.csv` is a list of roads with lengths, which is exactly Kruskal's input. No adjacency structure is needed.
3. **It is easy to explain and check by hand.** Kruskal processes roads from shortest to longest and makes one yes/no decision for each, as in the trace below. This matches how a planner would think: “lay the cheapest links first, unless the two places are already connected”.
4. **It handles a disconnected network gracefully.** If floods cut the network into pieces (test T18), Kruskal returns a minimum spanning forest, one tree per piece, without any change. Prim only builds the tree of the piece containing its start place.

We implemented **both** algorithms to confirm the answer. Prim gives the same 39.3 km total from every one of the 23 possible start places (test T24).

Kruskal trace on our graph

#	Road (sorted by length)	km	Decision
1	Mandala – QECH	1.0	add
2	QECH – Chichiri	1.2	add
3	Mpingwe – Limbe	1.2	add
4	Blantyre CBD – Sunnyside	1.3	add
5	Ginnery Corner – Chichiri	1.3	add
6	Ginnery Corner – Makata	1.4	add
7	Chichiri – Chitawira	1.4	add
8	Blantyre CBD – Mandala	1.5	add
9	Soche – Naperi	1.5	add
10	QECH – Ginnery Corner	1.5	skip: QECH and Ginnery Corner already connected (cycle)

#	Road (sorted by length)	km	Decision
11	Kanjedza – Chinyonga	1.5	add
12	Zingwangwa – Soche	1.6	add
13	Namiwawa – Ginnery Corner	1.6	add
14	Naperi – Chitawira	1.7	add
15	Blantyre CBD – Nyambadwe	1.8	add
16	Sunnyside – Soche	1.8	skip: Sunnyside and Soche already connected (cycle)
17	Chinyonga – Mpingwe	1.8	add
18	Blantyre CBD – Namiwawa	2.0	skip: Blantyre CBD and Namiwawa already connected (cycle)
19	Chilomoni – Kameza	2.0	add
20	Mandala – Naperi	2.0	skip: Mandala and Naperi already connected (cycle)
21	Ndirande – Makata	2.0	add
22	Blantyre CBD – QECH	2.2	skip: Blantyre CBD and QECH already connected (cycle)
23	Namiwawa – Ndirande	2.2	skip: Namiwawa and Ndirande already connected (cycle)
24	Manja – Zingwangwa	2.3	add
25	Kameza – Nyambadwe	2.4	add
26	Blantyre CBD – Chilomoni	2.5	skip: Blantyre CBD and Chilomoni already connected (cycle)
27	Nyambadwe – Ndirande	2.5	skip: Nyambadwe and Ndirande already connected (cycle)
28	Kanjedza – Limbe	2.5	skip: Kanjedza and Limbe already connected (cycle)
29	Mpingwe – Bangwe	2.5	add
30	Chilomoni – Manja	2.6	skip: Chilomoni and Manja already connected (cycle)
31	Makata – Kanjedza	2.8	add
32	Limbe – Bangwe	2.8	skip: Limbe and Bangwe already connected (cycle)
33	Ginnery Corner – Kanjedza	3.0	skip: Ginnery Corner and Kanjedza already connected (cycle)
34	Chichiri – Chinyonga	3.2	skip: Chichiri and Chinyonga already connected (cycle)
35	Chitawira – Chinyonga	3.4	skip: Chitawira and Chinyonga already connected (cycle)
36	Limbe – Machinjiri	3.5	add
37	Ndirande – Machinjiri	4.5	skip: Ndirande and Machinjiri already connected (cycle)

The tree is complete after 22 additions (the last is Limbe–Machinjiri, 3.5 km). The algorithm still scans the remaining longer roads but none can be added. Total: **39.3 km**.

Prim's trace from Blantyre CBD (for comparison)

#	Shortest road leaving the tree	km
1	Blantyre CBD → Sunnyside	1.3
2	Blantyre CBD → Mandala	1.5
3	Mandala → QECH	1.0
4	QECH → Chichiri	1.2
5	Chichiri → Ginnery Corner	1.3
6	Chichiri → Chitawira	1.4
7	Ginnery Corner → Makata	1.4
8	Ginnery Corner → Namiwawa	1.6

#	Shortest road leaving the tree	km
9	Chitawira → Naperi	1.7
10	Naperi → Soche	1.5
11	Soche → Zingwangwa	1.6
12	Blantyre CBD → Nyambadwe	1.8
13	Makata → Ndirande	2.0
14	Zingwangwa → Manja	2.3
15	Nyambadwe → Kameza	2.4
16	Kameza → Chilomoni	2.0
17	Makata → Kanjedza	2.8
18	Kanjedza → Chinyonga	1.5
19	Chinyonga → Mpingwe	1.8
20	Mpingwe → Limbe	1.2
21	Mpingwe → Bangwe	2.5
22	Limbe → Machinjiri	3.5

Total: **39.3 km**, the same tree as Kruskal, built in a different order.

8. Deployment

Q8 · 10 marks

The work is deployed in two ways, so it can be tested with nothing to install.

Web application (no installation)

The same Dijkstra, Kruskal and Prim algorithms are written in JavaScript in `docs/graph.js` and run in the browser. The page is hosted free on GitHub Pages:

https://mse23-mkapolo-cpu.github.io/DSA/

- **Shortest route:** choose any two places. The page shows the distance, the route highlighted on the map, and the full step-by-step table of the algorithm with the queue after each step.
- **Spanning tree:** shows the MST on the map with the Kruskal decision table, or Prim's order.
- **Tests:** runs the same test cases as the C++ suite in the browser and shows pass or fail for each.

C++ program

The C++ source is on GitHub at https://github.com/mse23-mkapolo-cpu/DSA. It can be run without installing anything by opening the repository in GitHub Codespaces (Code → Codespaces → Create), then running `make && ./roads` in the terminal. Alternatively, `src/graph.hpp` and `src/main.cpp` can be pasted into an online C++ compiler such as OnlineGDB. On a machine with g++ the steps are:

```
git clone https://github.com/mse23-mkapolo-cpu/DSA.git
cd DSA
make && make test
./roads path "Blantyre CBD" Limbe
```

Appendix A: src/graph.hpp

```
// graph.hpp - Weighted undirected graph of places in Blantyre city, with
// Dijkstra's shortest path and two Minimum Spanning Tree algorithms
// (Prim and Kruskal).
#pragma once

#include <algorithm>
#include <fstream>
#include <functional>
#include <limits>
#include <map>
#include <queue>
#include <sstream>
#include <stdexcept>
#include <string>
#include <tuple>
#include <vector>

namespace mw {

const long long INF = std::numeric_limits<long long>::max() / 4;

struct Edge {
    int u, v;
    long long w;
};

class Graph {
public:
    // Add a place if it is new; return its index either way.
    int addNode(const std::string& name) {
        auto it = index_.find(name);
        if (it != index_.end()) return it->second;
        int id = static_cast<int>(names_.size());
        names_.push_back(name);
        index_[name] = id;
        adj_.emplace_back();
        return id;
    }

    void addEdge(const std::string& a, const std::string& b, long long w) {
        if (w < 0) throw std::invalid_argument("Negative distance on road " + a + " - " + b +
                                               ": Dijkstra needs non-negative weights");
        if (a == b) throw std::invalid_argument("Self loop on " + a + " is not allowed");
        int u = addNode(a), v = addNode(b);
        adj_[u].push_back({v, w});
        adj_[v].push_back({u, w});
        edges_.push_back({u, v, w});
    }

    // Load "place_a,place_b,metres" lines. Blank lines and lines starting with # are skipped.
    static Graph fromCsv(std::istream& in) {
        Graph g;
        std::string line;
        int lineNo = 0;
        while (std::getline(in, line)) {
            ++lineNo;
            if (!line.empty() && line.back() == '\r') line.pop_back();
            if (line.empty() || line[0] == '#') continue;
            std::stringstream ss(line);
            std::string a, b, w;
            if (!std::getline(ss, a, ',') || !std::getline(ss, b, ',') || !std::getline(ss, w)) {
                throw std::runtime_error("Line " + std::to_string(lineNo) + ": expected place_a,place_b,metres");
            }
            long long metres;
            try {
                size_t used = 0;
                metres = std::stoll(w, &used);
                if (used != w.size()) throw std::invalid_argument("trailing");
            } catch (const std::exception&) {
                throw std::runtime_error("Line " + std::to_string(lineNo) + ": bad distance '" + w + "'");
            }
            g.addEdge(a, b, metres);
        }
        return g;
```

```
    }

    static Graph fromFile(const std::string& path) {
        std::ifstream f(path);
        if (!f) throw std::runtime_error("Cannot open " + path);
        return fromCsv(f);
    }

    int size() const { return static_cast<int>(names_.size()); }
    const std::vector<Edge>& edges() const { return edges_; }
    const std::string& name(int id) const { return names_.at(id); }
    const std::vector<std::pair<int, long long>>& neighbours(int u) const { return adj_.at(u); }

    int id(const std::string& name) const {
        auto it = index_.find(name);
        if (it == index_.end()) throw std::out_of_range("Unknown place: " + name);
        return it->second;
    }
    bool has(const std::string& name) const { return index_.count(name) > 0; }

private:
    std::vector<std::string> names_;
    std::map<std::string, int> index_;
    std::vector<std::vector<std::pair<int, long long>>> adj_;
    std::vector<Edge> edges_;
};

// ------------------------------------------------------------------ Dijkstra

struct DijkstraResult {
    std::vector<long long> dist;  // dist[v] = shortest distance in metres from source (INF if unreachable)
    std::vector<int> prev;        // prev[v] = previous place on the shortest path (-1 if none)
};

// One step of the algorithm, recorded for the dry run trace.
struct TraceStep {
    int settled;                                    // place removed from the queue and finalised
    long long settledDist;
    std::vector<std::tuple<int, long long, long long>> relaxed;  // (place, old dist, new dist)
};

// Dijkstra with a binary heap (std::priority_queue): O((V + E) log V).
inline DijkstraResult dijkstra(const Graph& g, int source, std::vector<TraceStep>* trace = nullptr) {
    int n = g.size();
    if (source < 0 || source >= n) throw std::out_of_range("Source index out of range");
    DijkstraResult r{std::vector<long long>(n, INF), std::vector<int>(n, -1)};
    std::vector<bool> done(n, false);
    using Item = std::pair<long long, int>;  // (distance, place)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;

    r.dist[source] = 0;
    pq.push({0, source});
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (done[u]) continue;  // stale queue entry
        done[u] = true;
        TraceStep step{u, d, {}};
        for (auto [v, w] : g.neighbours(u)) {
            if (done[v]) continue;
            if (d + w < r.dist[v]) {
                step.relaxed.emplace_back(v, r.dist[v], d + w);
                r.dist[v] = d + w;
                r.prev[v] = u;
                pq.push({r.dist[v], v});
            }
        }
        if (trace) trace->push_back(step);
    }
    return r;
}

// Rebuild the path source -> target from prev[]. Empty if unreachable.
inline std::vector<int> buildPath(const DijkstraResult& r, int target) {
    std::vector<int> path;
    if (r.dist.at(target) >= INF) return path;
    for (int v = target; v != -1; v = r.prev[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
```

```
    return path;
}

// ------------------------------------------------------------- MST

struct MstResult {
    std::vector<Edge> edges;
    long long total = 0;
    bool spanning = false;  // false if the graph is disconnected (then it is a forest)
};

// Disjoint Set Union with path compression and union by rank.
class DSU {
public:
    explicit DSU(int n) : parent_(n), rank_(n, 0) {
        for (int i = 0; i < n; ++i) parent_[i] = i;
    }
    int find(int x) { return parent_[x] == x ? x : parent_[x] = find(parent_[x]); }
    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (rank_[a] < rank_[b]) std::swap(a, b);
        parent_[b] = a;
        if (rank_[a] == rank_[b]) ++rank_[a];
        return true;
    }

private:
    std::vector<int> parent_, rank_;
};

// Kruskal: sort edges by weight, add each edge that joins two different components. O(E log E).
inline MstResult kruskal(const Graph& g) {
    MstResult r;
    std::vector<Edge> es = g.edges();
    std::stable_sort(es.begin(), es.end(), [](const Edge& a, const Edge& b) { return a.w < b.w; });
    DSU dsu(g.size());
    for (const Edge& e : es) {
        if (dsu.unite(e.u, e.v)) {
            r.edges.push_back(e);
            r.total += e.w;
        }
    }
    r.spanning = g.size() > 0 && static_cast<int>(r.edges.size()) == g.size() - 1;
    return r;
}

// Prim: grow one tree from `start`, always taking the cheapest edge leaving it. O(E log V).
inline MstResult prim(const Graph& g, int start = 0) {
    MstResult r;
    int n = g.size();
    if (n == 0) return r;
    std::vector<bool> inTree(n, false);
    using Item = std::tuple<long long, int, int>;  // (weight, from, to)
    std::priority_queue<Item, std::vector<Item>, std::greater<Item>> pq;
    pq.push({0, -1, start});
    while (!pq.empty()) {
        auto [w, from, to] = pq.top();
        pq.pop();
        if (inTree[to]) continue;
        inTree[to] = true;
        if (from != -1) {
            r.edges.push_back({from, to, w});
            r.total += w;
        }
        for (auto [v, wv] : g.neighbours(to))
            if (!inTree[v]) pq.push({wv, to, v});
    }
    r.spanning = static_cast<int>(r.edges.size()) == n - 1;
    return r;
}

}  // namespace mw
```

Appendix B: src/main.cpp

```
// main.cpp - Command line program for the Blantyre city road network
// (Limbe to Blantyre place). Distances are stored in metres and shown in km.
//
// Usage:
//   roads [--data FILE] path   <From> <To>    shortest route between two places
//   roads [--data FILE] trace  <From> <To>    same, plus a step-by-step dry run of Dijkstra
//   roads [--data FILE] all    <From>         shortest distance to every place
//   roads [--data FILE] mst    [prim|kruskal] minimum spanning tree
//   roads [--data FILE] places                list the places
//   roads [--data FILE]                      interactive menu
#include <iomanip>
#include <iostream>
#include <sstream>

#include "graph.hpp"

using namespace mw;

// Metres to km with one decimal, e.g. 1500 -> "1.5".
static std::string km(long long m) {
    if (m >= INF) return "inf";
    std::ostringstream o;
    o << std::fixed << std::setprecision(1) << m / 1000.0;
    return o.str();
}

static void printPath(const Graph& g, const std::string& from, const std::string& to, bool withTrace) {
    int s = g.id(from), t = g.id(to);
    std::vector<TraceStep> trace;
    DijkstraResult r = dijkstra(g, s, withTrace ? &trace : nullptr);

    if (withTrace) {
        std::cout << "Dry run of Dijkstra from " << from << "\n";
        int step = 1;
        for (const TraceStep& st : trace) {
            std::cout << "Step " << std::setw(2) << step++ << ": settle " << g.name(st.settled)
                      << " (" << km(st.settledDist) << " km)";
            if (st.relaxed.empty()) std::cout << ", no updates";
            for (auto [v, oldD, newD] : st.relaxed)
                std::cout << "\n          update " << g.name(v) << ": " << km(oldD) << " -> " << km(newD);
            std::cout << "\n";
            if (st.settled == t) {
                std::cout << "Target " << to << " settled, so its distance is final.\n";
                break;
            }
        }
        std::cout << "\n";
    }

    std::vector<int> path = buildPath(r, t);
    if (path.empty()) {
        std::cout << "No route from " << from << " to " << to << "\n";
        return;
    }
    std::cout << "Shortest route " << from << " -> " << to << ": " << km(r.dist[t]) << " km\n";
    for (size_t i = 0; i < path.size(); ++i) {
        std::cout << (i ? " -> " : "  ") << g.name(path[i]);
    }
    std::cout << "\n";
}

static void printAll(const Graph& g, const std::string& from) {
    DijkstraResult r = dijkstra(g, g.id(from));
    std::cout << "Shortest distances from " << from << ":\n";
    for (int v = 0; v < g.size(); ++v) {
        std::cout << "  " << std::left << std::setw(15) << g.name(v) << std::right << std::setw(6)
                  << km(r.dist[v]) << " km   via ";
        std::vector<int> p = buildPath(r, v);
        for (size_t i = 0; i < p.size(); ++i) std::cout << (i ? " > " : "") << g.name(p[i]);
        std::cout << "\n";
    }
}

static void printMst(const Graph& g, const std::string& algo) {
    MstResult m;
    if (algo == "prim") m = prim(g);
    else if (algo == "kruskal") m = kruskal(g);
```

```
    else throw std::invalid_argument("MST algorithm must be prim or kruskal");
    std::cout << "Minimum spanning tree (" << algo << "), in the order edges were added:\n";
    for (const Edge& e : m.edges)
        std::cout << "  " << std::left << std::setw(15) << g.name(e.u) << " - " << std::setw(15) << g.name(e.v)
                  << std::right << std::setw(5) << km(e.w) << " km\n";
    std::cout << "Edges: " << m.edges.size() << ", total length: " << km(m.total) << " km"
              << (m.spanning ? "" : "  (graph is disconnected: this is a spanning forest)") << "\n";
}

static void printPlaces(const Graph& g) {
    std::cout << g.size() << " places, " << g.edges().size() << " roads\n";
    for (int v = 0; v < g.size(); ++v) std::cout << "  " << g.name(v) << "\n";
}

static void interactive(const Graph& g) {
    while (true) {
        std::cout << "\n1) Shortest route  2) Route with dry run  3) Distances from a place\n"
                     "4) MST (Prim)  5) MST (Kruskal)  6) List places  0) Quit\nChoice: ";
        std::string c;
        if (!std::getline(std::cin, c) || c == "0") return;
        try {
            std::string a, b;
            if (c == "1" || c == "2" || c == "3") {
                std::cout << "From place: ";
                std::getline(std::cin, a);
            }
            if (c == "1" || c == "2") {
                std::cout << "To place: ";
                std::getline(std::cin, b);
            }
            if (c == "1") printPath(g, a, b, false);
            else if (c == "2") printPath(g, a, b, true);
            else if (c == "3") printAll(g, a);
            else if (c == "4") printMst(g, "prim");
            else if (c == "5") printMst(g, "kruskal");
            else if (c == "6") printPlaces(g);
            else std::cout << "Unknown choice\n";
        } catch (const std::exception& e) {
            std::cout << "Error: " << e.what() << "\n";
        }
    }
}

int main(int argc, char** argv) {
    std::vector<std::string> args(argv + 1, argv + argc);
    std::string data = "data/blantyre_roads.csv";
    if (args.size() >= 2 && args[0] == "--data") {
        data = args[1];
        args.erase(args.begin(), args.begin() + 2);
    }
    try {
        Graph g = Graph::fromFile(data);
        if (args.empty()) interactive(g);
        else if (args[0] == "path" && args.size() == 3) printPath(g, args[1], args[2], false);
        else if (args[0] == "trace" && args.size() == 3) printPath(g, args[1], args[2], true);
        else if (args[0] == "all" && args.size() == 2) printAll(g, args[1]);
        else if (args[0] == "mst") printMst(g, args.size() > 1 ? args[1] : "kruskal");
        else if (args[0] == "places") printPlaces(g);
        else {
            std::cerr << "Usage: roads [--data FILE] path|trace <From> <To> | all <From> | mst [prim|kruskal] |
places\n";
            return 2;
        }
    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << "\n";
        return 1;
    }
    return 0;
}
```

Appendix C: MST program output

```
$ ./roads mst kruskal
Minimum spanning tree (kruskal), in the order edges were added:
  Mandala         - QECH             1.0 km
```

```
  QECH            - Chichiri         1.2 km
  Mpingwe         - Limbe            1.2 km
  Blantyre CBD    - Sunnyside        1.3 km
  Ginnery Corner  - Chichiri         1.3 km
  Ginnery Corner  - Makata           1.4 km
  Chichiri        - Chitawira        1.4 km
  Blantyre CBD    - Mandala          1.5 km
  Soche           - Naperi           1.5 km
  Kanjedza        - Chinyonga        1.5 km
  Zingwangwa      - Soche            1.6 km
  Namiwawa        - Ginnery Corner   1.6 km
  Naperi          - Chitawira        1.7 km
  Blantyre CBD    - Nyambadwe        1.8 km
  Chinyonga       - Mpingwe          1.8 km
  Chilomoni       - Kameza           2.0 km
  Ndirande        - Makata           2.0 km
  Manja           - Zingwangwa       2.3 km
  Kameza          - Nyambadwe        2.4 km
  Mpingwe         - Bangwe           2.5 km
  Makata          - Kanjedza         2.8 km
  Limbe           - Machinjiri       3.5 km
Edges: 22, total length: 39.3 km
```